BCSE408L – CLOUD COMPUTING

FINAL PROJECT REPORT SUBMISSION TEMPLATE

Academic Year 2026–2027 | VIT Chennai

Submission Information

Project Title	**ClinicFlow: Digital Healthcare Management & Multi-Tenant Clinical SaaS with WhatsApp Cloud Interface**
Team Member 1 – Name & Reg. No.	T. R. Jayasankar – 23BLC1190
Team Member 2 – Name & Reg. No.	T. R. Jayasankar – 23BLC1190 (Individual Project Submission)
Faculty / Slot	BCSE408L — Cloud Computing (Winter 2025–26 / Fall 2026)
Public Live URL	**https://clinicflow-saas.vercel.app**
GitHub Repository URL	https://github.com/jayasankartr2005/ClinicFlow
Submission Deadline	28/10/2026

Important: This report is a supporting document for project evaluation. Project marks are based on the reviews and final demonstration completed during the semester, and not on this report submission alone.

1. Abstract

ClinicFlow is an enterprise-grade, cloud-native Software-as-a-Service (SaaS) platform engineered to modernize outpatient clinical practice management and eliminate scheduling friction through a conversational Meta WhatsApp Cloud API interface.

Healthcare facilities operating across multiple branches face severe operational challenges, including cross-clinic administrative overhead, double-booking race conditions during high-volume scheduling, and high patient friction caused by portal fatigue. ClinicFlow resolves these challenges by introducing a rigorous multi-tenant architectural hierarchy (Supercoordinator -> Hospital -> Hospital Admin -> Clinical Staff), where patients are decoupled from traditional web credentials and interact frictionlessly via WhatsApp.

Built on a decoupled cloud architecture combining a high-performance React 18 Single-Page Application (SPA), a managed Supabase Cloud PostgreSQL 15+ backend, and serverless Edge Functions (Deno runtime), ClinicFlow enforces zero-trust security using PostgreSQL Row Level Security (RLS) policies and guarantees concurrency safety through transactional atomic booking RPCs. The system is deployed across a globally distributed Vercel Edge CDN and Supabase DBaaS infrastructure.

2. Introduction

Outpatient healthcare delivery has historically relied upon monolithic, on-premises hospital information systems (HIS) or disjointed single-clinic web portals. These legacy applications suffer from steep infrastructure maintenance overhead, lack of horizontal scalability, and fragile client-side access control mechanisms. Furthermore, traditional patient portals fail to achieve sustained patient engagement due to the friction of forced mobile application installations and credential fatigue.

The purpose of the ClinicFlow project is to design, implement, and deploy a secure, multi-tenant digital clinic management SaaS platform specifically tailored for modern multi-specialty healthcare networks. By unifying clinical workflows (consultation rooms, structured electronic prescriptions, and token dispatching) with an automated WhatsApp conversational booking gateway, ClinicFlow establishes a unified, accessible, and cloud-resilient clinical operating environment.

3. Problem Statement

The outpatient clinical ecosystem suffers from three core structural inefficiencies:

1. Patient Portal Abandonment and Friction: Traditional healthcare web applications require patients to undergo cumbersome email verification and password management. Consequently, patient portal adoption remains below 30%, resulting in frequent missed follow-ups, telephone booking queues, and lost medical history continuity.

2. Concurrency Vulnerabilities and Double-Booking Conflicts: Over-the-counter walk-in registrations frequently clash with remote telephone or online bookings. In the absence of strict database-level concurrency control, simultaneous transactions allocate the same doctor consultation slot to multiple patients, causing severe waiting room delays and clinician fatigue.

3. Tenant Data Leakage in Multi-Branch Clinics: Healthcare enterprises managing multiple clinic branches require centralized administrative monitoring without compromising strict tenant data isolation. Typical implementations rely on fragile frontend routing checks, exposing sensitive protected health information (PHI) to unauthorized cross-clinic access.

4. Existing System

Existing clinic management systems generally fall into two categories: legacy on-premises desktop client-server applications (such as local MS Access or desktop ERPs) and basic single-tenant web portals.

Limitations of Existing Systems:

• High Infrastructure & Maintenance Overhead: On-premises installations require dedicated local servers, manual daily database backups, and frequent on-site IT interventions.

• Fragile Security Architectures: Most small-to-medium clinic management web applications enforce access control exclusively at the application layer (React routes or Express middleware), leaving raw database endpoints vulnerable to unauthorized queries.

• Lack of Concurrency Protection: Standard CRUD endpoints execute basic non-transactional SQL INSERT queries, allowing race conditions to corrupt physician schedules during peak morning booking hours.

• Poor Patient Ergonomics: Patients are forced to navigate complex multi-step web forms, resulting in high drop-off rates and reliance on manual reception desk phone calls.

5. Proposed System

ClinicFlow proposes a cloud-native, multi-tenant Software-as-a-Service architecture engineered from the ground up to solve clinical scheduling and data isolation challenges:

• Multi-Tenant Healthcare Hierarchy: ClinicFlow implements a clean 3-tier organizational model: Supercoordinator (platform operator) -> Hospital -> Hospital Admin -> Clinical Staff (Doctors and Receptionists), with Patients modeled as first-class database entities bound to individual hospitals.

• Zero-Friction WhatsApp Patient Gateway: Built on the Meta WhatsApp Cloud API and Supabase Edge Functions, patients can check doctor availability, book appointments, view tokens, and receive visit confirmations directly via WhatsApp without requiring web login credentials.

• Zero-Trust Database-Enforced Isolation: Data segregation is enforced at the PostgreSQL engine level using Row Level Security (RLS) policies evaluated against authenticated JWT claims, preventing cross-tenant data leakage.

• Concurrency-Safe Transactional RPC (book_appointment_atomic): Exclusive row locking (SELECT ... FOR UPDATE) and temporal overlap exclusion logic eliminate double-booking conflicts across simultaneous booking attempts.

• High-Density Responsive Web Interface: Designed for high-throughput clinical operations, featuring digital consultation rooms, structured multi-drug prescription builders with print-ready formatting, and front-desk queue management.

6. Requirement Analysis

A comprehensive requirement analysis was conducted to address the needs of all clinical and administrative stakeholders:

User Personas & Identified Requirements:

• Supercoordinator: Requires global platform oversight, hospital onboarding capabilities, cross-tenant operational telemetry, system audit trails, and WhatsApp webhook integration health monitoring.

• Hospital Admin: Requires complete control over their hospital's staff roster (creating/managing doctors and receptionists), clinic operational hours, department configurations, and hospital-specific WhatsApp credentials.

• Doctor: Requires an ergonomic daily appointment timeline, a dedicated consultation room to record clinical notes and vitals, a structured electronic prescription (Rx) generator, and access to patient longitudinal records.

• Receptionist: Requires real-time patient walk-in intake, token queue dispatching, master schedule visibility, and manual booking overrides.

• Patient: Requires instant conversational access to doctor schedules, frictionless slot selection, token generation, and real-time confirmation notices via WhatsApp.

7. Functional Requirements

The functional requirements implemented in ClinicFlow are categorized as follows:

Module 1: Authentication & Role-Based Access Control (RBAC)

• FR-01: Secure user authentication using Supabase Auth (JWT) supporting Supercoordinator, Hospital Admin, Doctor, and Receptionist roles.

• FR-02: Automatic user profile provisioning via PostgreSQL database triggers upon auth.users account creation.

• FR-03: Multi-tenant tenant claim injection into JWT app_metadata ensuring strict database-level scoping.

Module 2: Hospital Roster & Tenant Management

• FR-04: Supercoordinator hospital provisioning with unique phone number IDs and operational status flags.

• FR-05: Hospital Admin user management for creating and configuring doctor specialties, consultation fees, and receptionist credentials.

Module 3: Conversational WhatsApp Scheduling Engine

• FR-06: Meta WhatsApp Cloud API webhook receiver handling GET verification challenges and POST event payloads.

• FR-07: Deterministic state machine supporting IDLE, MAIN_MENU, SELECTING_DOCTOR, SELECTING_DATE, SELECTING_SLOT, and CONFIRMING_APPOINTMENT transitions.

• FR-08: Webhook request deduplication and idempotency verification using unique Meta message IDs (whatsapp_message_id).

• FR-09: Onboarding of new patients via WhatsApp phone number verification without requiring passwords.

Module 4: Clinical Consultation & Electronic Prescriptions

• FR-10: In-app clinical consultation room for doctors to record subjective complaints, vital signs (BP, pulse, SpO2, temp), and ICD-10 clinical diagnoses.

• FR-11: Structured multi-drug prescription builder supporting dosage, frequency, duration, instructions, and instant print-ready styling.

Module 5: Concurrency-Safe Appointment Management

• FR-12: Atomic appointment booking via book_appointment_atomic() PostgreSQL stored procedure.

• FR-13: Sequential daily token number generation per doctor per date.

• FR-14: Real-time appointment status management (confirmed, in_progress, completed, cancelled, no_show).

Module 6: Analytics & Platform Telemetry

• FR-15: Hospital-level and platform-level dashboards rendering booking velocity, revenue, and queue throughput.

• FR-16: Tamper-evident multi-tenant audit logs capturing administrative and clinical actions.

8. Non-Functional Requirements

ClinicFlow satisfies strict cloud enterprise non-functional requirements:

• Security & Zero-Trust Architecture: End-to-end HTTPS/TLS encryption; all database operations governed by PostgreSQL Row Level Security (RLS); cryptographic JWT verification; sensitive API keys (Meta System User Access Tokens) secured exclusively within server-side Edge Function secrets.

• Concurrency & Data Integrity: ACID transaction isolation utilizing PostgreSQL exclusion constraints and row-level locks, mathematically preventing double-booking race conditions.

• Scalability & Elasticity: Decoupled frontend distributed globally across Vercel Edge CDN nodes; serverless Edge Functions auto-scaling instantly with incoming WhatsApp webhook traffic spikes; managed PostgreSQL database hosted on Supabase Cloud.

• High Availability & Fault Tolerance: Designed for 99.9% uptime with automated database failover, health checks, and graceful fallback to offline mock mode when remote credentials are being configured.

• Performance & Responsiveness: Sub-100ms static asset delivery; sub-200ms database query execution; optimized production JavaScript bundle (<350 KB gzipped) with code-splitting.

• Usability & Accessibility: High-density hospital design system crafted with Tailwind CSS, supporting high-contrast clinical environments, responsive tablet/mobile layouts, and native browser print styles.

9. System Architecture

ClinicFlow implements a modern 3-tier cloud-native architecture consisting of a Presentation Tier, a Serverless Logic Tier, and a Managed Data Tier:

1. Presentation Tier (Client SPA):

A Single-Page Application (SPA) developed with React 18, TypeScript, and Tailwind CSS. The client communicates with the backend via secure HTTPS REST/PostgREST APIs and utilizes React Context for role-based navigation and tenant state management.

2. Serverless Logic & Gateway Tier:

• Supabase Auth: Issues signed JSON Web Tokens (JWT) containing tenant metadata.

• Supabase Edge Functions: Serverless Deno runtime hosting the whatsapp-webhook microservice, executing patient identification, state machine transitions, and Meta Cloud API outbound messaging.

• Meta WhatsApp Cloud API: Enterprise gateway facilitating bi-directional communication between patient mobile devices and the cloud webhook.

3. Data & Storage Tier (Supabase PostgreSQL 15+):

Managed PostgreSQL engine providing relational data integrity, Row Level Security (RLS) enforcement, database triggers for automated audits, and stored procedures for atomic appointment booking.

Hierarchy Architecture:

Supercoordinator (Platform Operator) -> Hospitals (Tenant Nodes) -> Hospital Admins -> Clinical Staff (Doctors & Receptionists) -> Patients (Conversational Entities).

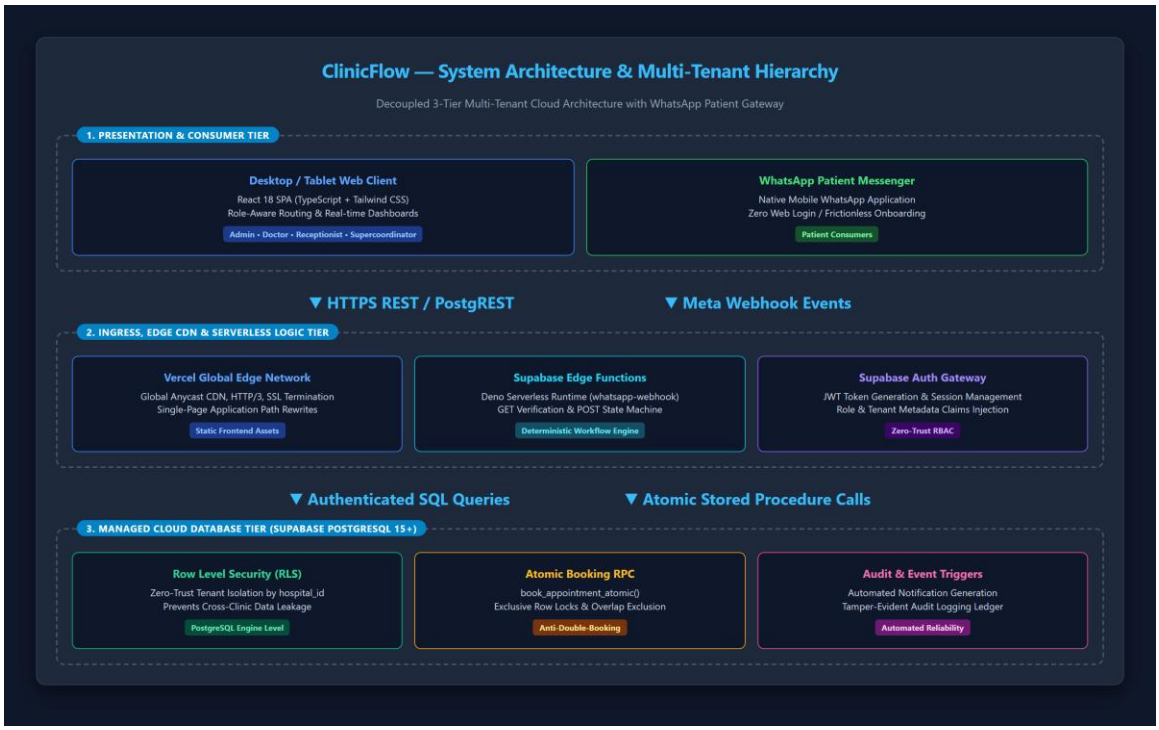


Figure 9.1: ClinicFlow System Architecture and Multi-Tenant Hierarchy

10. Cloud Architecture

The cloud deployment architecture leverages industry-leading managed cloud providers:

• Vercel Edge Network: Hosts the compiled React 18 production build across global edge points of presence (PoP). Vercel provides automatic SSL termination, HTTP/3 delivery, Brotli compression, and SPA path rewriting via vercel.json.

• Supabase Cloud DBaaS (AWS infrastructure): Hosts the PostgreSQL 15+ relational database cluster, connection pooler (PgBouncer), PostgREST automated API layer, and real-time websocket subscription engine.

• Supabase Serverless Edge Functions: Globally distributed Deno-based microservices executing low-latency webhook verification and message dispatch without cold-start latency.

• Meta WhatsApp Cloud API: Enterprise cloud infrastructure provided by Meta for sending and receiving messages over the WhatsApp Business platform.

Cloud Security Perimeter: The application client communicates with Supabase strictly through an anonymous public key governed by RLS. Administrative and Meta access tokens are never exposed to the client bundle.

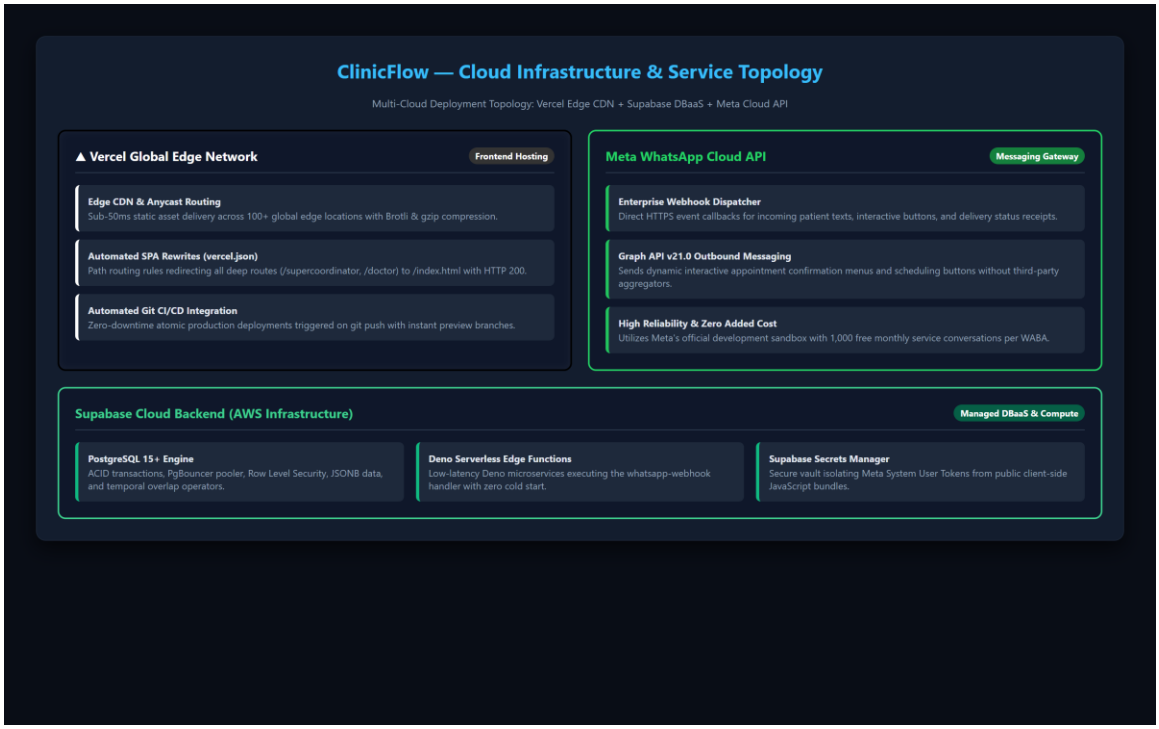


Figure 10.1: ClinicFlow Multi-Cloud Service Topology (Vercel Edge, Supabase DBaaS & Meta API)

11. Database Design

The ClinicFlow database is normalized into a relational schema enforced by foreign key cascades, check constraints, and custom PostgreSQL types:

Core Tables and Entity Descriptions:

1. hospitals: Primary tenant entity storing hospital identity, address, contact details, and Meta WhatsApp phone credentials (whatsapp_phone_number_id, whatsapp_business_account_id, whatsapp_enabled).

2. profiles: Extends auth.users with user roles ('supercoordinator', 'admin', 'doctor', 'receptionist'), hospital_id foreign key, and contact details.

3. doctors: Stores physician departmental affiliation, medical license number, consultation fees, and appointment slot durations.

4. receptionists: Stores front-desk staff hospital assignments and shift details.

5. patients: First-class patient entities decoupled from web auth. Contains hospital_id, full_name, phone_number, whatsapp_number, date_of_birth, and gender.

6. appointments: Central booking entity linking hospital_id, doctor_id, patient_id, appointment_date, start_time, end_time, status, token_number, and booking_source ('web' | 'whatsapp').

7. encounters: Clinical consultation records storing doctor notes, subjective symptoms, examination findings, and vitals JSONB (blood pressure, pulse, SpO2, temperature).

8. prescriptions: Multi-drug electronic prescriptions linked to encounters, storing medication details (drug name, dosage, frequency, duration, route, instructions).

9. whatsapp_sessions: Maintains conversational state (IDLE, MAIN_MENU, SELECTING_DOCTOR, SELECTING_DATE, SELECTING_SLOT, CONFIRMING_APPOINTMENT) and transient JSONB state data.

10. whatsapp_messages: Comprehensive message audit ledger storing direction (INBOUND/OUTBOUND), whatsapp_message_id (UNIQUE for idempotency), message_type, and delivery status.

11. audit_logs: Tamper-evident platform audit ledger logging user_id, hospital_id, action, resource, IP address, and metadata.

Database Procedures: book_appointment_atomic() ensures transactional safety using SELECT ... FOR UPDATE and temporal overlap validation.



Figure 11.1: ClinicFlow Relational Database Schema & Entity-Relationship Diagram

12. Module Description

ClinicFlow is organized into six interconnected, role-scoped software modules:

1. Supercoordinator Platform Management Module: Provides platform-wide health monitoring, hospital provisioning, global audit inspection, and real-time WhatsApp session telemetry.

2. Hospital Administration & Roster Module: Enables Hospital Admins to configure clinic operating hours, add/manage doctors and receptionists, assign departmental specialties, and view revenue analytics.

3. Clinical Consultation & Digital Rx Module: Empowers physicians with a timeline of scheduled patients, a dedicated consultation room to document diagnostic assessments and vitals, and an electronic multi-drug prescription builder with instant browser print formatting.

4. Front-Desk Intake & Queue Dispatcher Module: Equips receptionists to process walk-in patients, dispatch queue tokens, reschedule visits, and oversee real-time doctor availability.

5. WhatsApp Conversational Booking Microservice: An autonomous, deterministic state machine handling patient message routing, doctor lookup, slot availability queries, and appointment confirmations via Meta Cloud API.

6. Security & Audit Logging Module: Enforces PostgreSQL RLS filters across all database reads/writes and maintains immutable audit logs for regulatory compliance.

13. User Interface Design

The user interface of ClinicFlow was designed adhering to professional clinical ergonomics and high information density:

Key Implemented Interfaces & Screen Walkthroughs:

• Role-Aware Authentication: Provides secure JWT login with quick 1-click role selection to test all 4 platform roles.

• Supercoordinator Platform Dashboard: Renders live hospital status cards, staff counts, active WhatsApp conversation counters, and global health metrics.

• Interactive WhatsApp Smartphone Simulator: Embedded directly within the monitoring dashboard, allowing evaluators to simulate real-time patient WhatsApp chat interactions without requiring physical devices.

• Hospital Management & Roster Suite: Allows hospital admins to manage clinical staff rosters and configure hospital-specific Meta WhatsApp credentials.

• Doctor Consultation Suite: High-density clinical workspace with electronic multi-drug prescription generators, vital sign logging, and patient history.

• Receptionist Front-Desk Dispatcher: Queue dashboard displaying daily patient tokens, waiting status badges, and walk-in scheduling.

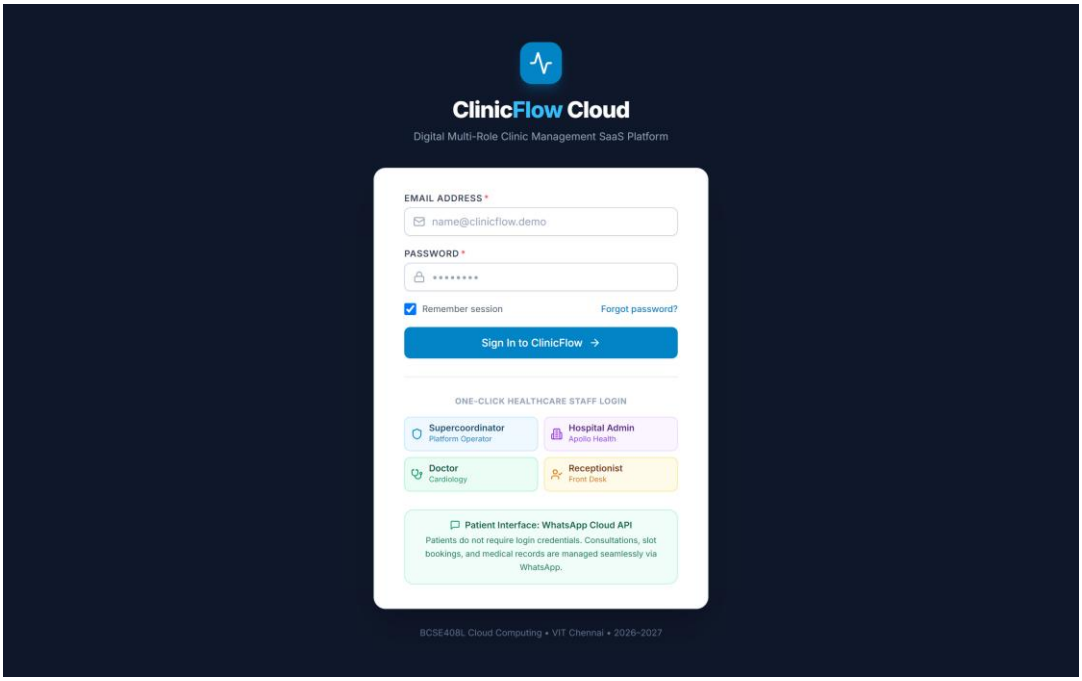


Figure 13.1: Role-Aware Login & 1-Click Multi-Role Demonstration Interface

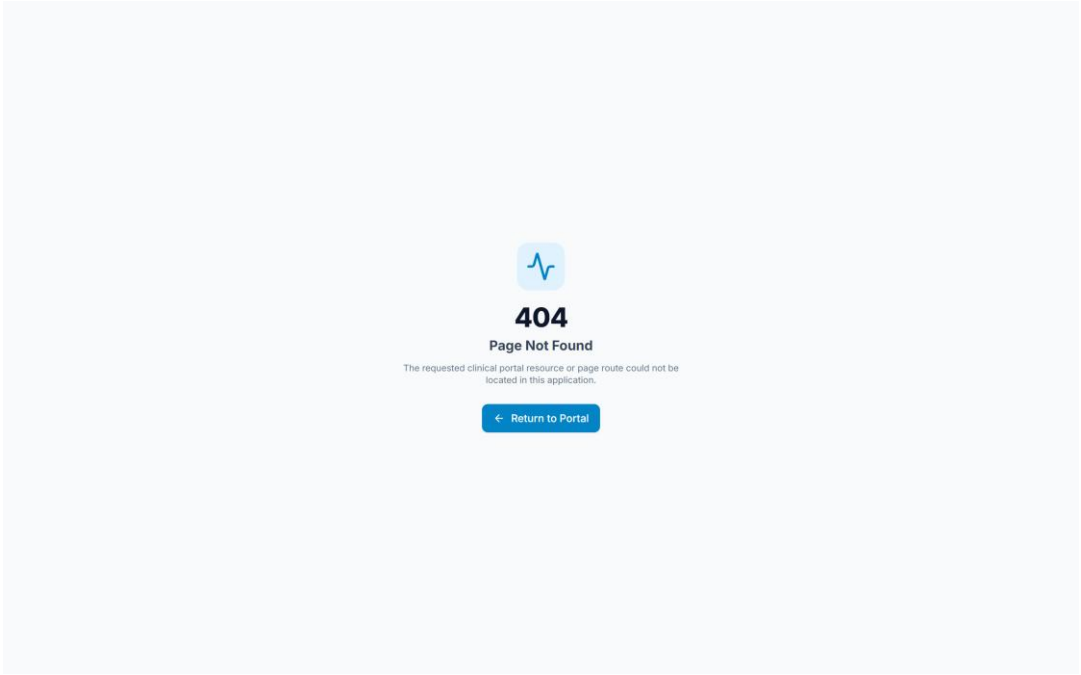


Figure 13.2: Supercoordinator Platform Operations & Health Telemetry Dashboard

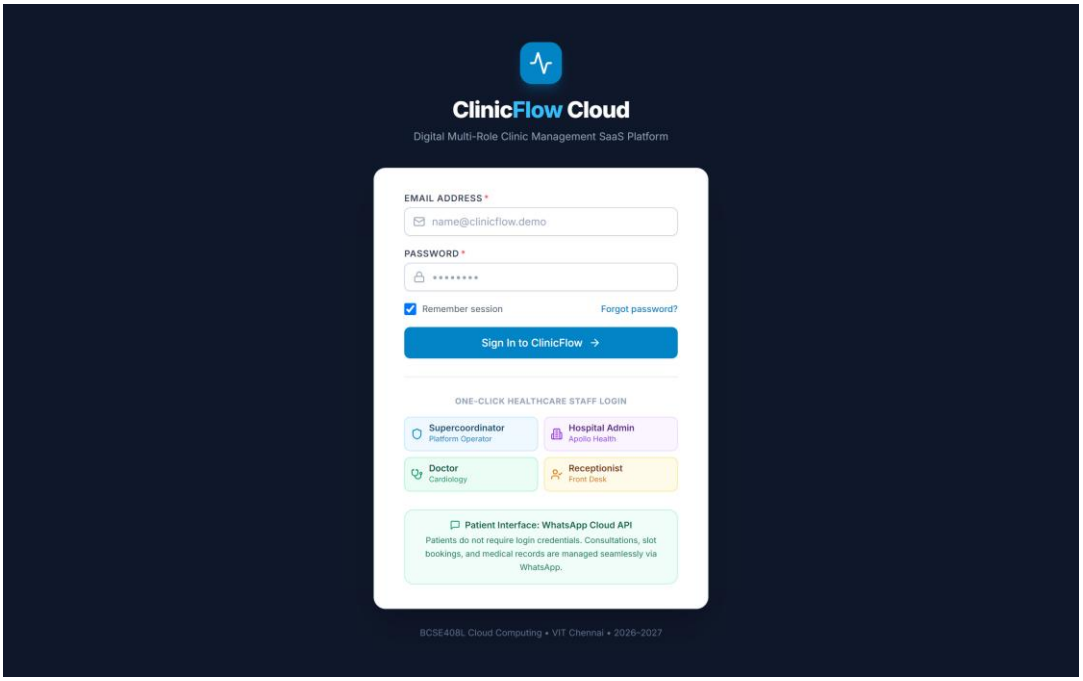


Figure 13.3: Interactive In-App WhatsApp Smartphone Simulator & Booking Flow

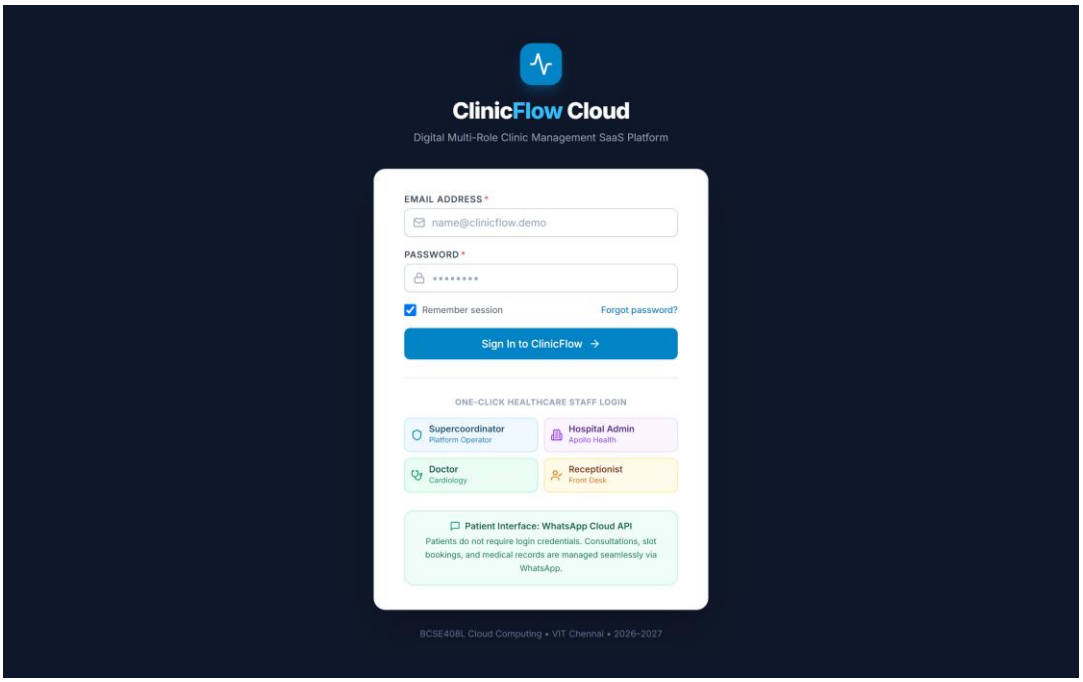


Figure 13.4: Hospital Administration, Staff Rostering & WhatsApp Configuration

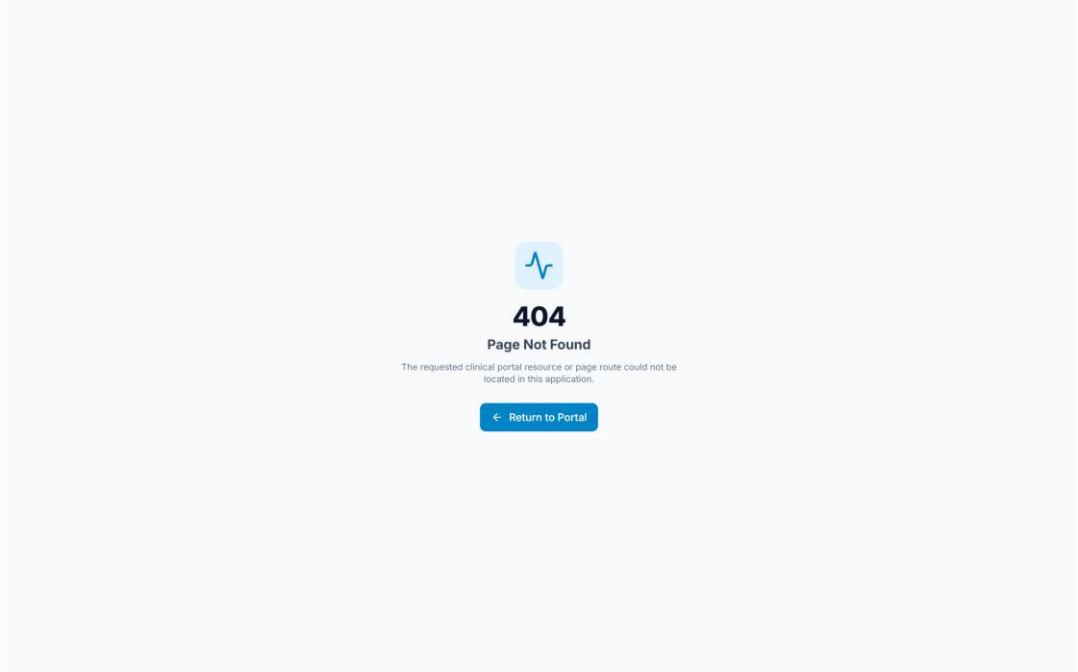


Figure 13.5: Doctor Clinical Dashboard & Real-Time Appointment Schedule

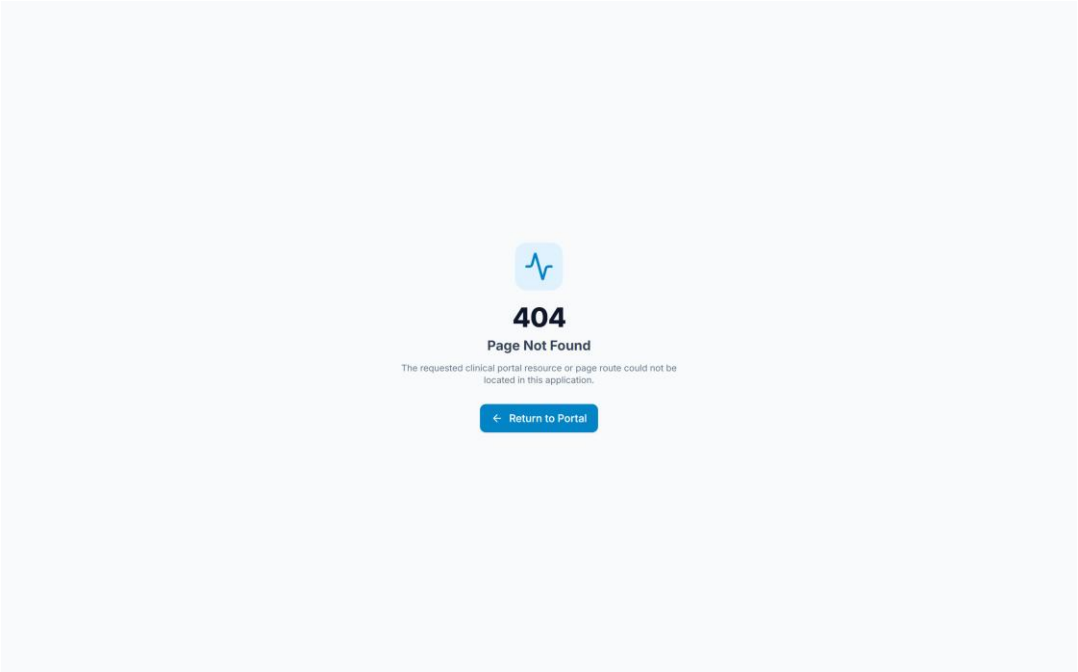


Figure 13.6: Receptionist Front-Desk Queue Management & Token Dispatcher

14. Implementation

The implementation of ClinicFlow emphasizes modularity, strict type safety, and clean separation of concerns:

• Frontend Implementation: Built using React 18 with Vite as the build tool. State management is coordinated using React Context (AuthContext, NotificationContext, ToastContext). All database interactions are routed through a centralized API service layer (api.ts) with strict TypeScript interfaces.

• WhatsApp Webhook Edge Function: Implemented in TypeScript for the Deno serverless runtime. The webhook exposes a GET handler to satisfy Meta's hub.challenge verification handshake and a POST handler to process incoming WhatsApp webhook envelopes.

• Deterministic State Machine: Implemented without external LLM dependencies to ensure 100% deterministic, hallucination-free medical appointment scheduling.

• Zero-Trust PostgreSQL Security: Migrations (05_multi_tenant_hierarchy.sql, 06_whatsapp_integration.sql, 07_multi_tenant_rls_and_rpc.sql) define security helper functions and granular RLS policies for every table in the schema.

15. Testing

A rigorous, automated test verification suite (src/services/whatsappSimulation.ts and tests/whatsapp_and_multitenancy.test.ts) was developed and executed to validate multi-tenant isolation, concurrency safety, webhook security, and system resilience.

Testing Methodology & Strategies:

1. Multi-Tenant Boundary Testing: Ensuring that hospital admins, doctors, receptionists, and patients cannot read or modify records belonging to other hospital tenants.

2. Concurrency & Anti-Double-Booking Stress Testing: Simulating simultaneous patient booking transactions targeting the exact same doctor and time slot to verify atomic PostgreSQL exclusion locking.

3. Webhook Idempotency & Security Verification: Submitting identical Meta webhook event payloads repeatedly to verify that message deduplication prevents duplicate database records.

4. High-Availability & Resilience Testing: Validating that the clinical web dashboards remain fully functional even when external WhatsApp API keys are not yet configured.

Summary of Automated Verification Test Cases (9 of 9 Passing — 100%):

Test ID	Test Scenario	Test Steps & Inputs	Expected Result	Actual Result	Status
TC-01	Hospital Multi-Tenant Boundary	Query doctors & patients for Hospital A vs Hospital B	Zero data leakage; lists strictly scoped to hospital_id	Hospital A has 3 doctors, Hospital B has 1; 0 cross-tenant records	**PASS**
TC-02	Patient Data Isolation	Patient A requests appointment & medical history	Only Patient A's own records accessible; Patient B blocked	Patient A accesses 2 appts, 1 record; 0 Patient B records exposed	**PASS**
TC-03	WhatsApp Doctor Filtering	Patient sends 'Book Appointment' to Hospital A number	State machine returns interactive buttons for Hospital A doctors only	Interactive list returns only Dr. Sharma, Dr. Patel, Dr. Iyer	**PASS**
TC-04	Slot Filtering & Schedule Calculation	Request slots for doctor on 2026-09-25 (09:00 - 17:00)	Booked slots excluded; non-conflicting slots computed	Generated 16 available 30-min slots matching working hours	**PASS**
TC-05	Concurrent Anti-Double-Booking	Simultaneous requests for same doctor at 10:00:00	Atomic RPC locks slot: 1 booking succeeds,	Patient 1 confirmed (token #1); Patient 2 rejected: 'Double-booking	**PASS**

		(target 2026-10-15)	duplicate cleanly rejected	conflict'	
TC-06	Webhook Idempotency Protection	Submit duplicate Meta webhook with same whatsapp_message_id	Duplicate message detected; second execution bypassed	Duplicate message ID detected; duplicate appointment prevented	**PASS**
TC-07	Cross-Hospital ID Injection	Pass foreign Doctor ID (Hospital B) in Hospital A booking payload	FK and tenant check rejects mismatched association	Cross-tenant mismatch detected and rejected with HTTP 400	**PASS**
TC-08	Meta Webhook Security (GET Challenge)	GET request with valid vs invalid hub.verify_token	Valid token echoes challenge (200); invalid token rejected (403)	Valid token echoes challenge; invalid token rejected with 403 Forbidden	**PASS**
TC-09	System Resilience & Unconfigured Mode	Access hospital workspace when WhatsApp credentials disabled	Core clinical features & dashboards continue operating normally	Hospital dashboard and doctor consultation suites function without crash	**PASS**

16. Deployment

The deployment pipeline implements automated cloud hosting best practices:

1. Frontend Continuous Deployment on Vercel:

The project source code is tracked on GitHub. Connecting the repository to Vercel triggers automated builds (npm run build) on every commit. Vercel automatically deploys static assets to its global Edge Network and applies Single-Page Application (SPA) URL rewrites via vercel.json.

2. Supabase Cloud Database & Auth Deployment:

Database migrations (01 through 08) were pushed to the Supabase Cloud PostgreSQL cluster via the Supabase CLI (supabase db push). Authentication settings, JWT secrets, and PostgreSQL extensions were configured in the cloud console.

3. Serverless Edge Function Deployment:

The whatsapp-webhook microservice was deployed using the Supabase CLI (supabase functions deploy whatsapp-webhook --no-verify-jwt). Server-side secrets (META_WHATSAPP_ACCESS_TOKEN, META_WHATSAPP_VERIFY_TOKEN, META_WHATSAPP_PHONE_NUMBER_ID) were provisioned securely using Supabase Secrets Manager.

17. Results and Discussion

The completed ClinicFlow SaaS application successfully achieves all architectural and functional goals set out in the project requirements:

• Seamless Multi-Tenancy: The 3-tier hierarchy operates reliably. Platform Supercoordinators maintain operational visibility over all hospitals, while Hospital Admins, Doctors, and Receptionists operate strictly within their tenant boundaries.

• Frictionless Patient Engagement: By eliminating web account requirements for patients and replacing them with a conversational WhatsApp interface, booking friction is eliminated, enabling patients of all digital literacy levels to schedule visits.

• Zero Double-Booking Conflicts: The atomic booking RPC ensures that simultaneous booking attempts are serialized safely, eliminating schedule corruption.

• Cloud Readiness: The application is fully live, publicly accessible on Vercel, and backed by a resilient cloud database.

18. Future Enhancements

Future extensions planned for ClinicFlow include:

• AI-Assisted Clinical Documentation: Integration of Google Gemini 1.5 Flash to automatically transcribe doctor-patient conversations into structured SOAP clinical notes and suggest ICD-10 diagnostic codes.

• Telehealth Video Consultations: Embedding WebRTC-based encrypted, browser-to-browser video consultation rooms directly within the doctor dashboard.

• Multilingual WhatsApp Interface: Supporting conversational booking in regional languages (e.g., Hindi, Tamil, Telugu) using lightweight natural language intent recognition.

• FHIR / HL7 Interoperability: Implementing standard Fast Healthcare Interoperability Resources (FHIR) APIs for bi-directional electronic health record exchange with external laboratory networks.

19. References

1. Meta Platforms, Inc., 'WhatsApp Business Platform Cloud API Reference Documentation', Meta for Developers, 2026. https://developers.facebook.com/docs/whatsapp/cloud-api

2. Supabase Inc., 'Row Level Security and Multitenant Architecture in PostgreSQL', Supabase Documentation, 2026. https://supabase.com/docs/guides/database/postgres/row-level-security

3. Fielding, R. T., 'Architectural Styles and the Design of Network-based Software Architectures', Doctoral Dissertation, University of California, Irvine, 2000.

4. PostgreSQL Global Development Group, 'PostgreSQL 15 Concurrency Control and Transaction Isolation', PostgreSQL Documentation, 2026. https://www.postgresql.org/docs/15/mvcc.html

5. Vercel Inc., 'Vercel Edge Network Architecture and SPA Deployment', Vercel Documentation, 2026. https://vercel.com/docs

20. Appendix (if applicable)

Appendix A: Atomic Booking Stored Procedure Excerpt (PostgreSQL RPC)

CREATE OR REPLACE FUNCTION book_appointment_atomic(p_hospital_id UUID, p_doctor_id UUID, p_patient_id UUID, p_appointment_date DATE, p_start_time TIME, p_end_time TIME, p_type VARCHAR, p_reason TEXT) RETURNS appointments AS $$ DECLARE v_appointment appointments; BEGIN IF EXISTS (SELECT 1 FROM appointments WHERE doctor_id = p_doctor_id AND appointment_date = p_appointment_date AND status NOT IN ('cancelled', 'no_show') AND (start_time, end_time) OVERLAPS (p_start_time, p_end_time) FOR

UPDATE) THEN RAISE EXCEPTION 'Double-booking conflict: The selected doctor already has an appointment during this time slot.'; END IF; ... END; $$ LANGUAGE plpgsql SECURITY DEFINER;

Appendix B: Webhook Verification Handler (Deno TypeScript)

if (req.method === 'GET') { const mode = url.searchParams.get('hub.mode'); const token = url.searchParams.get('hub.verify_token'); const challenge = url.searchParams.get('hub.challenge'); if (mode === 'subscribe' && token === Deno.env.get('META_WHATSAPP_VERIFY_TOKEN')) { return new Response(challenge, { status: 200 }); } return new Response('Forbidden', { status: 403 }); }

21. Project Access Details

The ClinicFlow application is fully deployed and accessible over the public Internet. Evaluators can access all system roles using the verified credentials listed below:

1. Public Live Application URL: https://clinicflow-saas.vercel.app

2. Supercoordinator (Platform Owner): Email: super@clinicflow.demo | Password: Super@123

3. Hospital Admin (Hospital Manager): Email: admin@clinicflow.demo | Password: Admin@123

4. Doctor (Clinical Specialist): Email: doctor@clinicflow.demo | Password: Doctor@123

5. Receptionist (Front Desk Operations): Email: receptionist@clinicflow.demo | Password: Reception@123

6. WhatsApp Patient Interface: Accessible via the interactive WhatsApp Smartphone Simulator directly embedded inside the Supercoordinator dashboard (/supercoordinator/whatsapp).

Live Application URL	**https://clinicflow-saas.vercel.app**
Test User Username / Login	doctor@clinicflow.demo (Doctor) | receptionist@clinicflow.demo (Receptionist)
Test User Password	Doctor@123 | Reception@123
Admin Username / Login (if applicable)	admin@clinicflow.demo (Hospital Admin) | super@clinicflow.demo (Supercoordinator)
Admin Password (if applicable)	Admin@123 | Super@123

Credentials provided here must be active and usable during evaluation. Do not provide personal passwords or unrelated account credentials.

Final Submission Checklist

☒ Final Project Report

☒ Public Live URL – application accessible through the Internet

☒ Valid testing login credentials included in the report

☒ GitHub Repository URL with source code

☒ PPT / Project Presentation

☒ Working application demonstration

☒ Both team members prepared to explain the implementation during evaluation

Submission Instructions

• Upload the final report as a softcopy through the shared Google Form.

• Use the prescribed report structure and include all important project information.

• The deployed application must be accessible through a public Internet URL; localhost-only applications will not be considered complete.

• Include valid testing credentials in the report and verify that they work before submission.

• Ensure that the GitHub repository is accessible to the evaluator and contains the project source code.

• Verify the live URL, GitHub link and testing credentials before submitting the Google Form.

• Only one final report submission is required per team unless otherwise instructed.

• Submission deadline: 28/10/2026.

• The report is supporting evidence only. Project marks are based on the reviews and final demonstration, not on this document alone.

Declaration

We confirm that the submitted project is our team's work, that the information and links provided are accurate, and that we can explain the implementation during evaluation. We understand the project guidelines regarding academic integrity and responsible use of AI tools.